

Information Retrieval

Chapter 17: Hierarchical Clustering

Complete Study Notes with Q&A

Based on: Introduction to Information Retrieval
Manning, Raghavan & Schütze

1. Introduction to Hierarchical Clustering

Flat clustering (Chapter 16) returns a plain set of clusters with no relationships shown between them. Hierarchical clustering builds a **tree-like structure** (hierarchy) that shows how clusters relate and merge with each other.

Problems with Flat Clustering

- **Unstructured output:** Just a set of clusters — no relationship between them.
- **Requires pre-specified K:** You must decide the number of clusters before running.
- **Nondeterministic:** Running twice may give different results (K-means depends on random init).

Advantages of Hierarchical Clustering

Advantage	Explanation
More informative output	A hierarchy shows structure and relationships between clusters
No need to pre-specify K	You don't decide the number of clusters in advance
Deterministic	Most hierarchical algorithms give the same result every time

Disadvantage: Efficiency

Hierarchical clustering is slower than flat clustering:

Algorithm	Complexity
K-means / EM (flat)	$O(n)$ — Linear — very fast
Hierarchical clustering	At least $O(n^2)$ — Quadratic — much slower

Two Types of Hierarchical Clustering

Type	Direction	How it works
Agglomerative (HAC)	Bottom-Up ■	Each doc starts as its own cluster → merge pairs → until one big cluster
Divisive	Top-Down ■	All docs in one cluster → split recursively → until individual documents

When to Use Which?

Use Flat Clustering when...	Use Hierarchical Clustering when...
Efficiency is the priority	You need richer structure in output
You already know K	You don't want to pre-specify K
Determinism doesn't matter	You need consistent, repeatable results

2. Section 17.1 — Hierarchical Agglomerative Clustering (HAC)

HAC = Bottom-up clustering. Start with N singleton clusters, merge the most similar pair step by step, until one big cluster remains. Requires exactly $N-1$ merges.

The Dendrogram

HAC is visualized using a **dendrogram** — a special diagram showing the entire merge history.

Element	Meaning
Each leaf (bottom)	A single document
Each horizontal line	A merge event between two clusters
Y-axis height of a line	The similarity at which those two clusters merged
Low horizontal line	Two very similar things merged early
High horizontal line (top)	Two less similar things merged late

■ Student: In the book it looks like a bar graph, not a tree. Why did you call it a tree?

■ Answer: Both descriptions are correct! A dendrogram is a tree drawn as a graph with a meaningful Y-axis. In the book (Figure 17.1), the dendrogram is rotated sideways — documents on the left, similarity axis going left-to-right — which is why it looks like a bar chart. Structurally, the merge history forms a binary tree.

■ Student: On the Y-axis — lower value = lower similarity clusters merged, higher value = higher similarity?

■ Answer: YES — exactly right! HAC merges the most similar pair FIRST (highest similarity = high on Y-axis). As merges continue, similarity decreases. So: High Y = early merge = high similarity. Low Y = late merge = low similarity. This is monotonicity in action.

Combination Similarity

Combination similarity = the Y-value (similarity) at which a merge happened. For a singleton cluster (single document), combination similarity = 1.0 (a document is perfectly similar to itself).

Monotonicity

A fundamental assumption in HAC: merge similarities must be **non-increasing**:

$$s_1 \geq s_2 \geq s_3 \geq \dots \geq s_{(K-1)}$$

An **inversion** ($s_i < s_{i+1}$) means a later merge had higher similarity than an earlier one — which violates the logic of always picking the best merge first.

How to Cut a Dendrogram

Strategy	Description
Largest gap	Cut where the biggest jump between successive similarities occurs — indicates natural clusters

Strategy	Description
Pre-specified similarity	Cut at a fixed Y value (e.g., $y=0.4$ gives 24 clusters, $y=0.1$ gives 12 clusters)
RSS + Penalty formula	$K = \operatorname{argmin}[RSS(K') + \lambda K']$ — balances cluster quality vs number of clusters
Pre-specify K	Just say 'I want exactly K clusters' and cut at that level

3. SimpleHAC Algorithm

The naive HAC algorithm works in two phases:

Why a 2D Array (Matrix)?

The algorithm needs to know: *'How similar is every document to every other document?'* The $N \times N$ matrix C stores all pairwise similarities as a lookup table so the algorithm can instantly find the best pair to merge at each step.

$C[i][j]$ = similarity between document i and document j . The diagonal $C[i][i] = 1.0$ always. The matrix is symmetric: $C[i][j] = C[j][i]$.

■ Student: Why are we creating a 2D array?

■ Answer: Because at every merge step the algorithm asks: 'Which two clusters are currently the most similar?' To answer instantly, it needs a pre-computed table of ALL pairwise similarities. That table IS the 2D array. Without it, similarities would need to be recalculated from scratch every step — even slower!

Algorithm Steps

Phase	What happens
Initialization	Build $N \times N$ similarity matrix C . Mark all N clusters as active ($I[n]=1$).
Main Loop ($N-1$ times)	Find the two active clusters with highest similarity. Record merge. Update row/column of merged cluster. Deactivate one cluster.
Return	Return list A of all merges in order.

Complexity: SimpleHAC is $O(N^3)$ — searching the full $N \times N$ matrix at each of $N-1$ steps.

4. Section 17.2 — Single-Link and Complete-Link Clustering

Single-Link Clustering

The similarity between two clusters = similarity of their **most similar (closest) pair** — one doc from each cluster. This is a **LOCAL** criterion — only cares about where two clusters come closest. Ignores all other documents.

Complete-Link Clustering

The similarity between two clusters = similarity of their **most dissimilar (farthest) pair** — one doc from each cluster. This is a **NON-LOCAL** criterion — the entire structure of both clusters influences the merge decision. Equivalent to choosing the merge with the **smallest diameter**.

Diameter = the distance/similarity between the two farthest documents inside a cluster.

Side-by-Side Comparison

Property	Single-Link	Complete-Link
Similarity measure	MAX similarity (closest pair)	MIN similarity (farthest pair)
Criterion type	Local	Non-local
Cluster shape tendency	Long, straggly clusters	Compact, tight clusters
Main problem	Chaining	Sensitivity to outliers
Merge decision based on	One closest pair only	Entire cluster structure

■ Student: I didn't understand what complete-link is doing. Why does it pick the docs that are farthest in each cluster?

■ Answer: Complete-link is NOT finding docs farthest from their own cluster center. It looks at pairs ACROSS two clusters being considered for merging, and picks the pair that is farthest apart from each other. The idea is: 'I will only merge two clusters if I can GUARANTEE that ALL documents in the merged cluster are reasonably similar.' It checks the worst-case pair — if even the worst pair is acceptable, then definitely all other pairs are fine too.

■ Student: So we merge those docs which are furthest away from their centroids?

■ Answer: No! Complete-link has nothing to do with centroids. It finds the farthest pair of documents — one from each cluster being considered for merging. Centroids are used in a completely different algorithm: Centroid Clustering (Section 17.4).

Chaining — Single-Link's Problem

Chaining: A chain of documents gets merged one by one into a single long, straggly cluster because each new document is close to ONE existing member — even if far from most others. Like a chain of people holding hands: Person1–Person2–Person3–...–Person6. Each is close to their neighbor, but Person1 and Person6 are far apart. Single-link puts them all in one cluster.

Outlier Sensitivity — Complete-Link's Problem

One outlier document far from the cluster center can completely change the final clustering. The outlier inflates the diameter of candidate merge clusters, blocking natural merges from happening.

Shared Weakness of Both Methods

Both methods reduce cluster quality to just ONE pair of documents — either the closest or the farthest. One pair cannot fully represent the distribution of all documents in a cluster. This is why Group-Average clustering (Section 17.3) was introduced — it considers ALL pairs instead.

■ Student: What is the problem in single-link and complete-link?

■ Answer: The core problem: both look at only ONE pair out of potentially thousands. Say Cluster A has 100 docs and Cluster B has 100 docs — there are 10,000 pairs. Single-link looks at only the 1 closest pair. Complete-link looks at only the 1 farthest pair. Both ignore 9,999 pairs completely. This single pair cannot represent the true quality of a potential merge.

■ Student: What is the benefit of complete-link over single-link and when to use which?

■ Answer: Use Single-Link when: clusters have irregular/non-globular shapes, data has no outliers, loosely connected groups are acceptable. Use Complete-Link when: compact tight clusters are needed, balanced cluster sizes are important, data is clean with no outliers. Neither is perfect — both look at only one extreme pair, which is their shared weakness.

Graph-Theoretic Interpretation

Method	Clusters at step k are...
Single-link	Connected components of $G(s_k)$ — groups where any point is reachable from any other via at least one link
Complete-link	Maximal cliques of $G(s_k)$ — groups where EVERY point is directly connected to EVERY other point

5. Section 17.2.1 — Time Complexity

Algorithm	Complexity	Key Idea
SimpleHAC	$O(N^3)$	Exhaustive search of full matrix at each step
EfficientHAC	$O(N^2 \log N)$	Priority queues keep similarities sorted — finding max is instant
SingleLinkClustering	$O(N^2)$	NBM array tracks best merge per cluster — no sorting needed

EfficientHAC — Priority Queue Algorithm

Each cluster gets its own **priority queue** — a sorted list of similarities to all other clusters, highest first. Finding the best merge = just look at the top of each queue.

Two separate steps happen:

- **DECIDE step:** Look at the TOP of ALL priority queues → pick the globally highest similarity pair → that's k_1 and k_2 to merge.
- **UPDATE step:** After merging k_1 and k_2 , for every other cluster i : delete old entries for k_1 and k_2 , compute new similarity with merged cluster, insert back.

■ **Student:** Why are we merging inside the same priority queue? We compare the highest similarity one with other priority queue's top and then merge them.

■ **Answer:** You are correct! The merge decision always happens ACROSS different priority queues — compare tops of all queues, find the globally highest pair. The update (what the P[5] example showed) happens WITHIN each remaining cluster's priority queue to reflect the newly merged cluster. These are two completely separate steps. Sorry for the earlier confusion!

Similarity Update Formulas

Method	Formula for $\text{sim}(i, \text{merged}\{k_1, k_2\})$	Plain English
Single-link	$\max(\text{sim}(i, k_1), \text{sim}(i, k_2))$	Take the closer of the two old similarities
Complete-link	$\min(\text{sim}(i, k_1), \text{sim}(i, k_2))$	Take the farther of the two old similarities
Centroid	$(1/N_m \times v_m) \cdot (1/N_i \times v_i)$	Dot product of average vectors
Group-average	Complex formula	Average of all pairwise similarities

■ **Student:** What is 'i' in the update formula?

■ **Answer:** 'i' = any OTHER cluster that is NOT part of the current merge. When k_1 and k_2 merge, the algorithm must update the similarity of the new merged cluster $\{k_1+k_2\}$ with every remaining cluster. 'i' is each of those remaining clusters, one at a time.

NBM Array — Single-Link Optimization

NBM = Next Best Merge. An array where $\text{NBM}[i]$ = the cluster that is currently the best merge candidate for cluster i . Instead of searching the whole matrix, each cluster keeps a personal note of its current best friend. Updates are just simple max comparisons — no sorting needed $\rightarrow O(N^2)$.

Best-Merge Persistence

Single-link has a special property: **best-merge persistence**. If cluster ω_j is the best merge candidate for ω_k , then after ω_j merges with another cluster ω_i , the merged cluster $\{\omega_i, \omega_j\}$ is STILL ω_k 's best merge candidate.

Why? Because single-link uses MAX: $\text{sim}(k, \text{merged}\{i, j\}) = \max(\text{sim}(k, i), \text{sim}(k, j)) \geq \text{sim}(k, j)$. So the new similarity is at least as good as before. ■

Complete-link does NOT have this property — it uses MIN, so merging a faraway cluster can dramatically reduce similarity and change best-merge candidates unexpectedly. This is why only single-link can use NBM.

Practical Note

Even though single-link is $O(N^2)$ and others are $O(N^2 \log N)$, in practice the difference is negligible because computing document similarity (e.g., dot product) costs far more than sorting. All four HAC algorithms are effectively **$O(N^2)$ with respect to similarity computations**.

■ Student: So the one which is merged first (k_1) will stay alive?

■ Answer: Yes! Exactly right. k_1 = stays ACTIVE and becomes the representative of the merged cluster. k_2 = gets DEACTIVATED ($I[k_2] \leftarrow 0$) and disappears. All future references use k_1 to represent the merged cluster. Think of it like a company acquisition: if Apple absorbs Nokia, Nokia ceases to exist — only Apple (k_1) remains.

6. Full Worked Example — EfficientHAC (Single-Link)

4 documents: d1, d2, d3, d4. Similarity matrix:

	d1	d2	d3	d4
d1	1.0	0.9	0.2	0.3
d2	0.9	1.0	0.7	0.4
d3	0.2	0.7	1.0	0.6
d4	0.3	0.4	0.6	1.0

Initialization: Priority Queues

```
P[1]: d2(0.9) → d4(0.3) → d3(0.2)    ← d1's best friend = d2
P[2]: d1(0.9) → d3(0.7) → d4(0.4)    ← d2's best friend = d1
P[3]: d2(0.7) → d4(0.6) → d1(0.2)    ← d3's best friend = d2
P[4]: d3(0.6) → d2(0.4) → d1(0.3)    ← d4's best friend = d3
```

Iteration 1

DECIDE: Tops: $P[1]=0.9$, $P[2]=0.9$, $P[3]=0.7$, $P[4]=0.6 \rightarrow \text{Max} = 0.9 \rightarrow \text{Merge } \mathbf{d1 \text{ and } d2}$

UPDATE P[3]: Delete d1(0.2) and d2(0.7). New sim = $\max(0.2, 0.7) = 0.7$. Insert d1(0.7).

P[3] after: d1(0.7) → d4(0.6)

UPDATE P[4]: Delete d1(0.3) and d2(0.4). New sim = $\max(0.3, 0.4) = 0.4$. Insert d1(0.4).

P[4] after: d3(0.6) → d1(0.4)

Iteration 2

DECIDE: Tops: $P[1]=0.7$, $P[3]=0.7$, $P[4]=0.6 \rightarrow \text{Max} = 0.7 \rightarrow \text{Merge } \mathbf{\{d1+d2\} \text{ and } d3}$

UPDATE P[4]: Delete d3(0.6) and d1(0.4). New sim = $\max(0.4, 0.6) = 0.6$. Insert d1(0.6).

P[4] after: d1(0.6)

Iteration 3

DECIDE: Only two clusters left: $\{d1+d2+d3\}$ and d4 $\rightarrow \text{Merge them at similarity } 0.6$

Final merge list A:

- Merge 1: d1 + d2 $\rightarrow$ similarity 0.9
- Merge 2: $\{d1+d2\} + d3 \rightarrow$ similarity 0.7
- Merge 3: $\{d1+d2+d3\} + d4 \rightarrow$ similarity 0.6

Monotonicity holds: $0.9 \geq 0.7 \geq 0.6$ ■

7. Quick Reference Summary

All Algorithms — Complexity

Algorithm	Complexity	Can use NBM?	Best-merge persistent?
SimpleHAC	$O(N^3)$	No	N/A
EfficientHAC	$O(N^2 \log N)$	No	Depends on method
SingleLinkClustering	$O(N^2)$	Yes	Yes
Complete/Group-Avg/Centroid	$O(N^2 \log N)$	No	No

Common Student Confusions

Confusion	Clarification
Does hierarchical clustering need K?	NO — that's one of its key advantages over flat clustering
Higher Y on dendrogram = more similar?	YES — higher Y = earlier merge = higher similarity
Complete-link uses centroids?	NO — centroids are a separate algorithm (Section 17.4)
Single-link uses MAX — isn't that picking the worst merge?	No! MAX similarity = CLOSEST pair = BEST connection between clusters
k2 stays alive after merge?	No — k1 stays alive as representative, k2 is deactivated
P[5] example showed merging inside same queue?	No — that showed the UPDATE step after a merge decided elsewhere